

INSTITUTO FEDERAL

Paraíba

Campus Campina Grande

ENGENHARIA DE COMPUTAÇÃO

Sistemas Embarcados - *Alexandre Sales Vasconcelos*

Lavoisier Chaves Ramos - Mat.: 202311250006

abril - 2026

Exercícios - 6

Controle de Eventos Críticos com Interrupções e Temporizadores

Objetivo

Refatorar o sistema da Atividade 05 para que o processador não precise “vigiar” o botão o tempo todo. O sistema deve ser despertado apenas quando o evento físico ocorrer, utilizando Interrupções de Hardware (ISR). Realizar o tratamento do efeito *bounce* por software.

Material Necessário

- 1 ESP32 (DevKit)
- 1 LED + 1 Resistor de $220\ \Omega$
- 1 Botão + 1 Resistor de $10\ k\Omega$ (opcional, se usar *pull-up* interno)
- Protoboard e cabos

Passos para a Atividade

Desenvolvimento do firmware

Desenvolva um programa utilizando o ESP-IDF para implementar um controlador de acionamento de um LED.

Requisitos

- **Primeiro Clique:** Liga o LED e inicia a contagem (10 segundos).
- **Cliques Subsequentes:** Se o LED já estiver ligado, cada novo clique do botão deve reiniciar o temporizador de 10 segundos (como um sensor de presença de corredor que renova o tempo ao detectar movimento).
- **Desligamento Forçado:** Se o usuário segurar o botão por mais de 2 segundos, o LED deve apagar imediatamente, ignorando o temporizador.

Diagrama em blocos

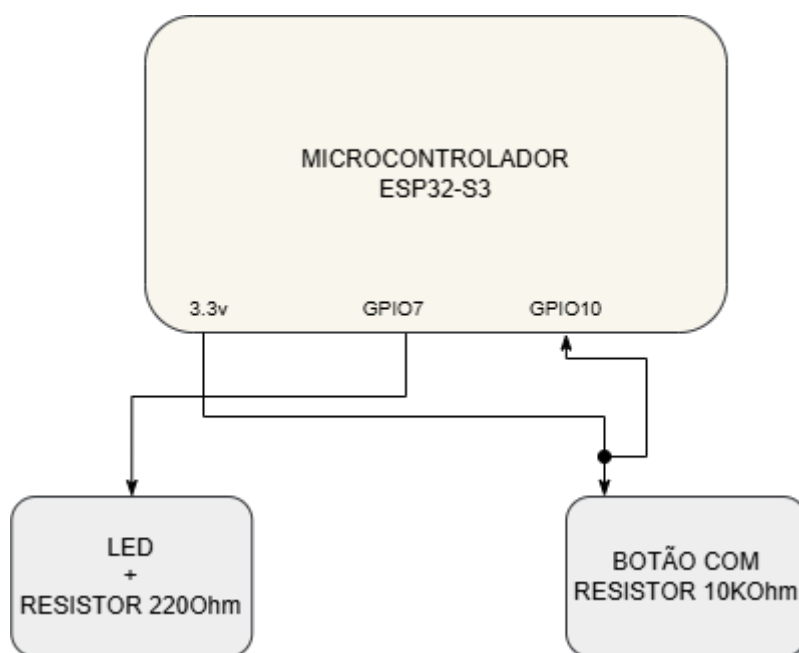
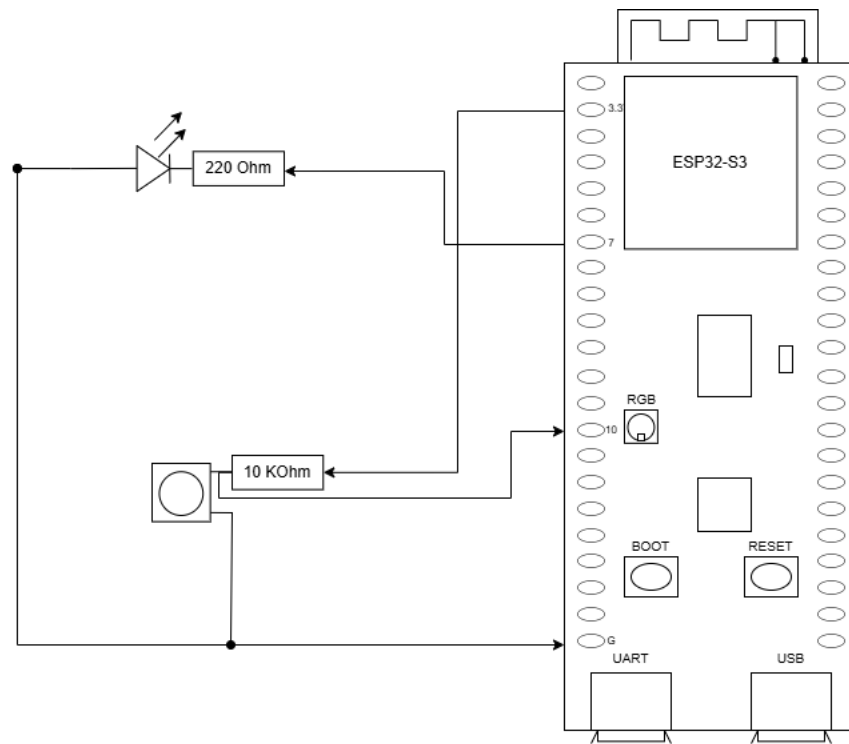
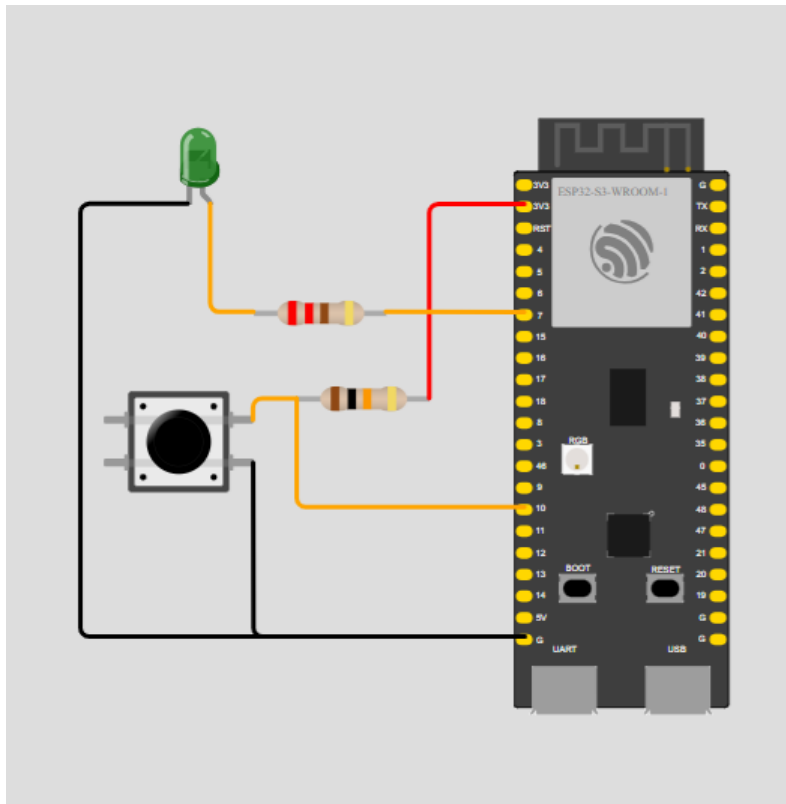


Diagrama esquemático



Simulação do circuito no Wokwi



Meu circuito no Wokwi

Desenvolvimento do código em C para firmware comentado.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_timer.h"

#define BTN GPIO_NUM_10
#define LED GPIO_NUM_7

#define DEBOUNCE_TIME 50000 // 50 ms
#define HOLD_TIME 2000000 // 2 s
#define LED_TIMEOUT 10000000 // 10 s

static int64_t last_isr_time = 0;
static esp_timer_handle_t timer_led;
```

```

static esp_timer_handle_t timer_hold;

volatile int hold_detected = 0;

// Callback: Desliga o LED após 10s de inatividade
void timer_led_callback(void* arg) {
    gpio_set_level(LED, 0);
}

// Callback: Executado se o botão for segurado por 2s
void timer_hold_callback(void* arg) {
    hold_detected = 1;
    gpio_set_level(LED, 0);          // Apaga imediatamente
    esp_timer_stop(timer_led);      // Cancela qualquer contagem de 10s ativa
}

static void IRAM_ATTR button_isr_handler(void* arg) {
    int64_t now = esp_timer_get_time();
    if (now - last_isr_time < DEBOUNCE_TIME) return;
    last_isr_time = now;

    int level = gpio_get_level(BTN);

    if (level == 0) { // BOTÃO PRESSIONADO
        hold_detected = 0;

        // Ação imediata: Liga o LED e para o timer de desligamento atual
        gpio_set_level(LED, 1);
        esp_timer_stop(timer_led);

        // Inicia a contagem de 2s para ver se o usuário vai segurar
        esp_timer_start_once(timer_hold, HOLD_TIME);
    } else { // BOTÃO SOLTO
        // Para o timer de 2s, pois o botão foi solto antes ou depois do hold
        esp_timer_stop(timer_hold);

        // Se soltou e NÃO era um hold (foi um clique curto ou renovação)
        if (!hold_detected) {
            // Inicia ou reinicia os 10 segundos para apagar
            esp_timer_stop(timer_led);
        }
    }
}

```

```

        esp_timer_start_once(timer_led, LED_TIMEOUT);
    }
}

}

void app_main(void) {
    // Configura LED
    gpio_config_t io_conf = {
        .mode = GPIO_MODE_OUTPUT,
        .pin_bit_mask = (1ULL << LED),
        .intr_type = GPIO_INTR_DISABLE,
        .pull_down_en = 0,
        .pull_up_en = 0
    };
    gpio_config(&io_conf);

    // Configura botão com Pull-up Externo
    gpio_config_t btn_conf = {
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE, // Pull-up interno desativado
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_ANYEDGE,    // Monitora tanto o aperto quanto a soltura
        .pin_bit_mask = (1ULL << BTN)
    };
    gpio_config(&btn_conf);

    // Criação dos Timers
    esp_timer_create_args_t led_args = { .callback = &timer_led_callback, .name = "LED Timer" };
    esp_timer_create_args_t hold_args = { .callback = &timer_hold_callback, .name = "Hold Timer" };

    esp_timer_create(&led_args, &timer_led);
    esp_timer_create(&hold_args, &timer_hold);

    // Gera interrupções
    gpio_install_isr_service(0);
    gpio_isr_handler_add(BTN, button_isr_handler, NULL);

    while (1) {
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

Com o Wokwi, foi gerado um arquivo .json com o código abaixo, representando a montagem da simulação do circuito.

```
{
  "version": 1,
  "author": "Lavoisier Chaves Ramos",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-s3-devkitc-1",
      "id": "esp",
      "top": 0,
      "left": 0,
      "attrs": { "builder": "esp-idf" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 121.4,
      "left": -182.4,
      "attrs": { "color": "black", "xray": "1" }
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -3.6,
      "left": -159.4,
      "attrs": { "color": "green", "flip": "" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 80.75,
      "left": -115.2,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 119.15,
```

```
    "left": -96,
    "attrs": { "value": "10000" }
  }
],
"connections": [
  [ "esp:TX", "$serialMonitor:RX", "", [ ] ],
  [ "esp:RX", "$serialMonitor:TX", "", [ ] ],
  [ "btn1:2.r", "esp:GND.1", "black", [ "h0.2", "v77" ] ],
  [ "led1:A", "r1:1", "orange", [ "v0" ] ],
  [ "led1:C", "esp:GND.1", "black", [ "h-47.6", "v192.18" ] ],
  [ "r2:2", "esp:3V3.2", "red", [ "v0" ] ],
  [ "r2:1", "btn1:1.r", "orange", [ "h-19.4" ] ],
  [ "r1:2", "esp:7", "orange", [ "v0" ] ],
  [ "r2:1", "esp:10", "orange", [ "v0" ] ]
],
"dependencies": {}
}
```


Protótipo do circuito.

